

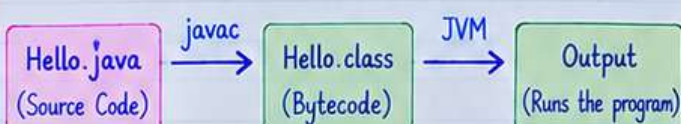
Maven Deep Dive

Topics : JAR Files, Maven, pom.xml, Dependencies, Repositories and Maven Lifecycle

Goal : Understand why Maven exists and how it helps Java projects manage structure, dependencies, builds and packaging.

1 The Basic Journey of Java Code

- A Java program goes through compilation before it is executed.



Key Points :

- ★ .java file → source code written by the developer
- ★ .class file → compiled bytecode understood by the JVM
- ★ Java code is not directly executed as .java file.
- ★ First it is compiled into .class files, then JVM runs those files.

2 What Happens in a Real Java Project ?

- A real project usually has many Java classes.

Example files :

- User.java
- Order.java
- Payment.java
- EmailService.java
- InvoiceGenerator.java
- Main.java

After compilation :

- User.class
- Order.class
- Payment.class
- EmailService.class
- InvoiceGenerator.class
- Main.class

Problem :

- ★ Sharing many .class files is not practical.
 - ★ Files can get misplaced.
 - ★ Package structure can break.
 - ★ Some files may be missed (resources, configs etc.).
 - ★ Running the project becomes confusing.
- Java needed a standard way to bundle everything together.

3 What is a JAR File ?

- JAR stands for Java Archive.
- A JAR file is a packaged bundle for Java projects (similar to a ZIP file).
- It contains compiled code and related resources.

A JAR file can contain :

- .class files
- folders / packages
- images
- properties files
- configuration files
- metadata

In simple words :

"A JAR file is a standard way to package Java compiled code and related resources into one file."

4 Why Do We Need a JAR File ?

Reason 1: To Share Java Code Easily

- Suppose you create a useful calculator library.
- Instead of sending all compiled .class files separately, you can package them into:

calculator.jar

- Now another developer can add this JAR to their project and use your compiled code.
- This is how many Java libraries are shared.

Reason 2: To Use External Libraries

- When we use libraries like:

- Spring Core
- MySQL Connector
- Hibernate
- Jackson

- We are actually using someone else's JAR files.
- For example, when we say "I am using Spring Core", our project is using Spring Core JAR files.

5 Library vs Application

It is important to understand the difference between a library and an application.

Term	Meaning
Library	Code created for other developers to use inside their projects. It usually does not run independently.
Application	Code created for the end user. It can run as a complete program, usually with a main() method or an application entry point.

Example :

Library JAR

calculator.jar

- Provides reusable calculator functions.
- Does not run independently.

Application JAR

myapp.jar

- Contains a complete runnable program.
- Has a main() method.

Maven Deep Dive

6 Normal Library JAR vs Spring Boot Executable JAR

Normal Library JAR

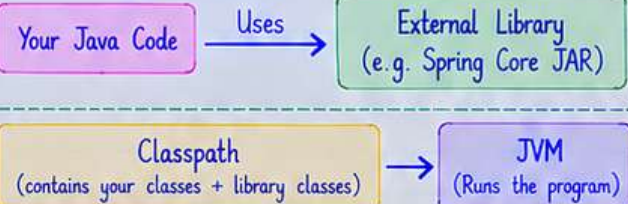
- Usually contains only your compiled code.
- Does not include all external dependency JARs inside it.
- Dependencies are managed separately using Maven (`pom.xml`).
- Used as a library in other projects.

Spring Boot Executable JAR

- Contains application code + required dependencies.
- Can run as a single JAR file.
- Called a **fat JAR** (or executable JAR).
- Useful for deployment.
- Created using Spring Boot build process (Maven).

7 External Libraries and Classpath

- Suppose your Java code uses an external library (e.g. Spring Core).
- The library classes are inside a JAR file.
- To use those classes, the JAR must be available in the classpath.
- Classpath tells the JVM where to find the `.class` files (both your code and external libraries).
- Maven automatically downloads the required JAR files and manages the classpath for us.



8 What is Maven ?

- Maven is a build tool for Java projects.
- It helps in:
 - Project structure
 - Managing dependencies
 - Building the project
 - Running tests
 - Packaging the application (JAR/WAR)
 - Deployment
- Maven uses a configuration file called `pom.xml`.

In simple words :

"Maven is a tool that automates and standardizes the way we build Java projects."

9 Why Do We Need Maven ?

- Without Maven, we have to manually download JAR files, manage classpath, and handle project structure.
- Maven solves these problems by:
 - Automatically downloading dependencies from online repositories.
 - Managing correct versions.
 - Providing a standard project structure.
 - Giving simple commands for build, test and package.

Key Benefits :

- ★ No manual download of JAR files.
- ★ Automatic dependency management.
- ★ Consistent project structure.
- ★ Standard build process.
- ★ Easier collaboration in teams.

10 What is pom.xml ?

- `pom.xml` (Project Object Model) is the main configuration file in a Maven project.
- It contains information about:
 - Project details (`groupId`, `artifactId`, `version`)
 - Dependencies
 - Build configuration
 - Plugins
 - Properties
 - Packaging type (`jar/war`)
 - And more...

Location :

It is present in the root folder of the Maven project.

Note :

Everything in Maven is driven by `pom.xml`.

11 Important Tags in pom.xml

1. <groupId>

- Uniquely identifies the project (like a package name).
- Usually follows reverse domain name.
- Example: `com.coderarmy`

Example :

```
<groupId>
  com.coderarmy
</groupId>
```

2. <artifactId>

- Name of the project or module.
- Example: `myapp`

Example :

```
<artifactId>
  myapp
</artifactId>
```

3. <version>

- Specifies the project version.
- Example: `1.0.0`

Example :

```
<version>
  1.0.0
</version>
```

Maven Deep Dive

12 <dependencies> Tag

- Used to declare external libraries (dependencies) required for the project.
- Maven automatically downloads these dependencies from online repositories (like Maven Central).
- Each dependency has :

- groupId
- artifactId
- version

- Example :

```
<dependencies>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>3.2.0</version>
</dependency>
</dependencies>
```

Note :

Maven handles transitive dependencies automatically.

13 Maven Repositories

- A repository is a storage location where JAR files and other artifacts are kept.
- When we add a dependency in pom.xml, Maven searches the repository, downloads the required JAR files and stores them in our local machine.
- Common repositories :

Repository	Description
Maven Central	Default and most commonly used repository (https://repo1.maven.org)
Local Repository	Stored in your machine (~/.m2/repository)
Remote Repository	Hosted on the internet (e.g. Maven Central, company repositories)

How it works ?



14 Transitive Dependencies

- A dependency can have its own dependencies. Maven automatically downloads those as well.
- This is called transitive dependency.
- Example :
If we add spring-boot-starter-web, Maven will also download many other required libraries (e.g. spring-core, spring-web, etc.).

16 Build Output Location

- When we run a Maven build (e.g. mvn package), the output JAR file is created inside the target folder.
- Example :

```
myapp/
├── target/
│   └── myapp-1.0.0.jar
```

Note :

The target folder is created during build and can be deleted using mvn clean.

15 Maven Plugins

- Plugins provide specific functionality during the build process.
- Example : spring-boot-maven-plugin helps to create an executable JAR in Spring Boot.
- Plugins are configured in the <build> section of pom.xml.

Example :

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

18 Key Takeaways

- A JAR file packages compiled Java code and resources.
- Maven helps manage project structure, dependencies, builds and more.
- pom.xml is the heart of a Maven project.
- Dependencies are downloaded from repositories.
- Maven uses transitive dependencies.
- The build lifecycle has multiple phases (validate → deploy).
- For Spring Boot, an executable JAR (fat JAR) is created.
- Common commands like mvn clean and mvn package are used frequently.

17 Common Maven Commands

Command	Purpose
mvn clean	Deletes the target folder.
mvn compile	Compiles the source code.
mvn test	Runs unit tests.
mvn package	Creates the JAR/WAR file.
mvn install	Installs the artifact into local repository.
mvn deploy	Deploys the artifact to a remote repository.
mvn spring-boot:run	Runs the Spring Boot application.

19 Quick Recap Diagram

